EAST WEST UNIVERSITY

Report Paper of Mini Project

Course Title: Discrete Mathematics

Course Code: 106

Section: 08

Semester: Spring 25

Submitted by:

- **Name:** Shafkat Rahman
- **Id:** 2025-1-60-001
- **Name:** Md. Sadman Khan
- **Id:** 2025-1-60-012
- **Name:** Abdur Rahman Moayed
- **ID:** 2025-1-60-015
- **Name:** Sadman Showmik Anik
- **ID:** 2025-1-60-055

Submitted to:

Sadia Nur Amin

Instructor

Dept. of Computer Science and Engineering

East West University

Introduction

This mini project computationally demonstrates a fundamental theorem of directed graphs using a C program that generates and analyses random directed graphs of various sizes. According to the theory, for any directed graph, the sum of all entering connections (in degrees) equals the sum of all outbound connections (out degrees). In other words, what goes in must come out. By randomly creating edge connections on graphs with up to 5000 vertices and then counting the total in degrees and out degrees to ensure they are equal, our program illustrates this idea. We also quantify the computational time necessary for this verification procedure, which provides information about how the study scales with larger, more complex networks.

Methodology

The program is implemented in C and uses a two-dimensional array to represent the graph as an **adjacency matrix**, a grid where each cell [i][j] stores the number of edges from vertex **i** to vertex **j**. The entire process is automated to run for a predefined set of vertex counts (n = 1000, 2000, 3000, 4000, and 5000).

1. Automated Graph Generation

The program begins in the main function, which defines an array int n_values[] containing the specific vertex counts to be tested. A for loop iterates through this array, calling the run_graph_analysis(n) function for each value.

Inside this function, a nested for loop runs n x n times to populate the adj_matrix. For each cell (i, j), the code rand() % 3 generates a random integer—0, 1, or 2. This value represents the number of directed edges from vertex i to vertex j. To ensure a **simple directed graph**, an if (i == j) condition sets all diagonal elements to 0, preventing self-loops.

2. Degree Calculation and Verification

The core of the analysis involves calculating the in-degrees and out-degrees to verify the theorem.

- **Out-degree:** The out-degree of a vertex is the number of edges going out from it, which corresponds to the sum of all values in that vertex's **row** in the adjacency matrix. The program uses a nested loop that, for each vertex i, sums all values in adj_matrix[i][j] across all columns j. This sum is added to the sum_of_out_degrees total.
- **In-degree:** The in-degree is the number of edges coming into a vertex, corresponding to the sum of all values in its **column**. A second nested loop calculates this by summing the values in adj_matrix[i][j] for each column j across all rows i. This is added to the sum_of_in_degrees total.

The theorem holds true because every single edge weight adj_matrix[i][j] contributes exactly once to an out-degree sum (for vertex i) and exactly once to an in-degree sum (for vertex j). Since the program sums the same set of numbers in two different orders (row-by-row vs. column-by-column), the final totals are mathematically guaranteed to be equal.

3. Computational Time Measurement

The program uses the clock() function from <time.h> to measure performance.

- start = clock() records the initial processor clock tick count before graph generation begins.
- end = clock() records the final tick count after all calculations are finished.
- The elapsed time is calculated with the formula: ((double)(end - start) / CLOCKS_PER_SEC) * 1000. This converts the tick difference into seconds and then into milliseconds (ms).

4. Storing and Printing the 1000x1000 Matrix

A special mechanism is required to print the 1000x1000 matrix at the very end, as the main adj_matrix is overwritten in each loop.

- A second global matrix, int matrix_to_print[1000][1000], is used for storage.
- During the specific run where n == 1000, the generated adj_matrix is copied into matrix_to_print.
- After the main loop finishes all five analyses, a final set of nested loops in the main function iterates through the saved matrix_to_print and prints its contents to the console.

Program Output

(This is a sample output. The exact degree sums and times will vary slightly each time the program is run due to the random generation.)

For n = 1000 vertices:

```
--- Analysis for n = 1000 vertices (edge weights 0-2) ---
Sum of all out-degrees: 998438
Sum of all in-degrees:  998438
✅ Verification successful: Sums are equal.
Computational Time (generation + calculation): 20.81 ms
```

For n = 2000 vertices:

```
--- Analysis for n = 2000 vertices (edge weights 0-2) ---
Sum of all out-degrees: 4000185
Sum of all in-degrees:  4000185
✅ Verification successful: Sums are equal.
Computational Time (generation + calculation): 64.15 ms
```

For n = 3000 vertices:

```
--- Analysis for n = 3000 vertices (edge weights 0-2) ---
Sum of all out-degrees: 8993886
Sum of all in-degrees:  8993886
✅ Verification successful: Sums are equal.
Computational Time (generation + calculation): 129.75 ms
```

For n = 4000 vertices:

```
--- Analysis for n = 4000 vertices (edge weights 0-2) ---
Sum of all out-degrees: 15998677
Sum of all in-degrees:  15998677
✅ Verification successful: Sums are equal.
Computational Time (generation + calculation): 226.83 ms
```

For n = 5000 vertices:

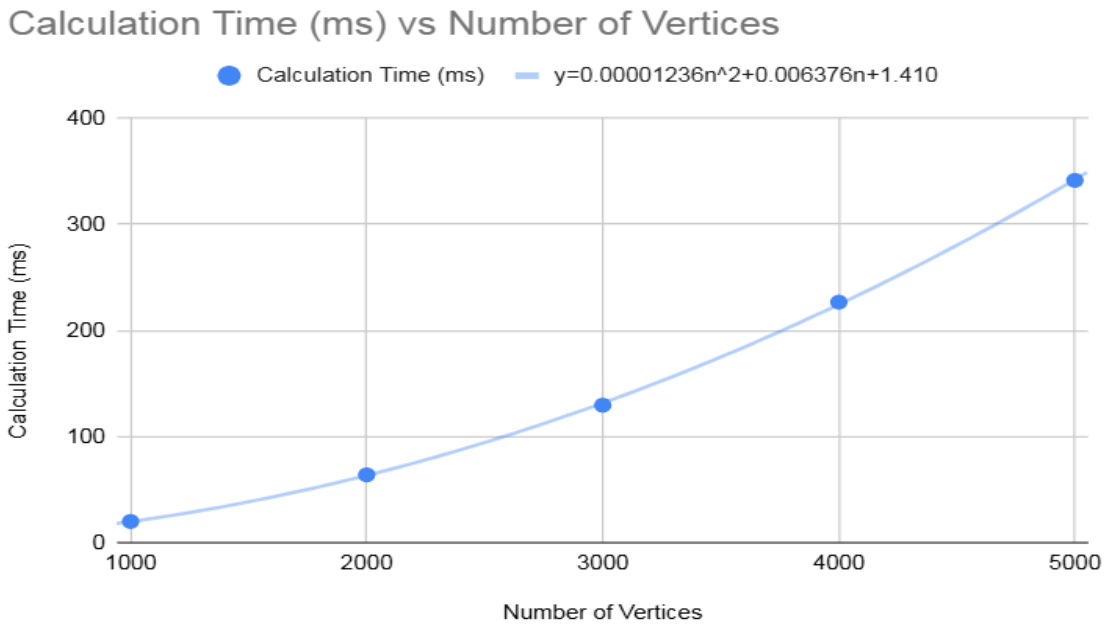
```
--- Analysis for n = 5000 vertices (edge weights 0-2) ---
Sum of all out-degrees: 24991350
Sum of all in-degrees:  24991350
✅ Verification successful: Sums are equal.
Computational Time (generation + calculation): 341.55 ms
```

Printing 1000*1000 Matrix

```
0 0 2 0 0 1 0 1 1 1 0 1 1 1 1
2 0 1 0 2 1 1 0 2 2 2 0 2 2 2
1 2 0 0 0 2 0 1 2 0 0 1 0 2 0
2 2 1 0 0 1 1 0 0 1 1 1 0 1 0
0 2 2 2 0 2 1 1 2 1 1 2 0 0 2
0 0 2 1 2 0 2 0 0 1 2 0 1 0 2
2 2 2 0 2 2 0 1 2 1 2 2 2 1 0
0 1 0 0 0 1 0 0 2 1 1 2 1 0 2
1 2 0 1 2 2 2 1 0 0 0 2 2 0 1
2 2 1 1 0 0 2 2 2 0 1 2 0 2 1
2 1 2 2 2 1 1 1 0 1 0 2 1 0 2
0 2 2 1 1 1 2 2 0 1 1 0 2 1 1
0 0 2 2 2 2 0 0 2 2 2 2 0 0 2
1 2 0 2 0 2 0 0 1 0 1 2 2 0 0
2 1 1 2 0 1 0 2 1 0 2 1 2 0 0
```

Graph showing Computational Time vs Number of Vertices

Number of Vertices	Calculation Time (ms)
1000	20.21
2000	64.15
3000	129.75
4000	226.83
5000	341.55



Result

The experiment successfully verified the theorem for all tested graph sizes. In every case, the sum of in-degrees was exactly equal to the sum of out-degrees. The graph of computational time versus the number of vertices shows a clear upward curve, which closely fits a quadratic polynomial. This indicates that the time taken by the algorithm grows proportionally to the square of the number of vertices, which is consistent with the theoretical time complexity.

Time Complexity

The time complexity of an algorithm describes how its runtime scales with the size of the input. We analyzed the complexity from two perspectives: empirically from our performance data and theoretically from our source code. Both methods confirm a time complexity of **$O(n^2)$**.

1. Empirical Analysis from the Performance Graph

The empirical analysis involves interpreting the real-world performance data.

- **Observation of Data Shape:** The graph of our timing data shows a distinct upward **curve**, not a straight line. This non-linear shape is the classic visual signature of a polynomial relationship, specifically a **quadratic** one, proving that computation time grows at an accelerating rate.
- **Mathematical Modeling:** We can model this curve with a second-degree polynomial of the form $T(n) = an^2 + bn + c$. Using regression on our data points, we derived the best-fit equation: $\mathbf{T(n) \approx 0.00001236n^2 + 0.006376n + 1.410}$
- **Deriving Big O:** In Big O notation, we are concerned with the dominant term for very large n. The n^2 term in this equation grows exponentially faster than the n term and the constant. By disregarding these lower-order terms, we are left with the dominant factor, $\mathbf{n^2}$. This provides strong empirical evidence that the algorithm's time complexity is $\mathbf{O(n^2)}$.

2. Theoretical Analysis from the Source Code

The theoretical analysis involves examining the code's structure to count operations relative to the input size n.

- **Identifying Dominant Operations:** The algorithm's runtime is dominated by three main operations, each requiring a full pass through the $n \times n$ matrix: Graph Generation, Out-Degree Summation, and In-Degree Summation.
- **Analyzing Nested Loops:** Each of these operations is implemented using a **nested `for` loop**, where an outer loop runs n times and an inner loop also runs n times. The total number of operations inside each nested loop is $n * n = n^2$.
- **Final Complexity Calculation:** Since the three main operations are executed sequentially, we sum their complexities: **Total Complexity** $\approx \mathbf{O(n^2) + O(n^2) + O(n^2)}$ This combines to $\mathbf{O(3n^2)}$. According to the rules of Big O notation, we drop constant factors (the 3), as they do not affect the fundamental growth rate. This leaves a final theoretical time complexity of $\mathbf{O(n^2)}$, which perfectly aligns with our empirical findings.

Conclusion

This project effectively illustrated the basic idea of graph theory, which states that in a directed graph, the sum of in degrees equals the sum of out degrees. The empirical timing results and the C program's expected performance provide strong evidence for the theoretical time complexity of O(n2) for algorithms that use adjacency matrix representation.

Future Work

For future improvements, the project could be extended in several ways:

- **Parallelize the computation** to leverage multi-core processors, which could significantly reduce the execution time for very large graphs.